Business Intelligence

Proyecto final · Entrega 2

Data warehouse, ETL y tablero

MoviBogotá S.A. · Data warehouse de validaciones y tablero de demanda

Integrante	Código
David Santiago Prieto Beltrán	506212024
Naren Santiago Rojas Sánchez	506232716
Joseph David Vargas Olaya	506232072

Dato	Valor
Docente	Leandro Pájaro Fuentes
Fecha de entrega	06/11/2026
Repositorio	https://github.com/Narens-dev/bi-proyecto-equipo506232716
Motor de base de datos	PostgreSQL 16
Herramienta del tablero	Looker Studio (Google Data Studio)

Resumen de la entrega

Se construyó el data warehouse **movibogota_dw** en PostgreSQL 16 con un modelo en estrella de **7 tablas** (6 dimensiones y la tabla de hechos fact_validaciones). Un único trabajo de Pentaho Data Integration (j_carga_dw.kjb) ejecuta 7 transformaciones que leen las tres fuentes en su formato original (F1 validaciones en CSV, F2 Estaciones Troncales de TRANSMILENIO en CSV y F3 Trazados Troncales de TRANSMILENIO en GeoJSON), aplican 13 reglas de limpieza y cargan **54.775 registros en la tabla de hechos** (de 55.000 leídos; 225 descartados por duplicados o montos no válidos). La carga se concilia por fuente y por total de recaudo, y alimenta un tablero de Looker Studio con 3 páginas y 4 indicadores.

Respuesta a P0: la troncal **Autopista Norte** es la de mayor demanda, con **10.141 validaciones (18,5 %) y $29.442.500 de recaudo**, y su estación Portal Norte - Unicervantes es la de más validaciones de la red (2.058), con el máximo en la franja pico de la mañana (618). Las dos franjas pico concentran el 56,6 % de las validaciones, por encima de la meta de 55 %.

Ajustes frente a la retroalimentación de la Entrega 1: (1) las preguntas P0, P2 y P3 se replantearon para que se respondan con los datos del inventario; (2) F2 y F3 son ahora conjuntos de datos específicos con URL directa y formato único (F2 CSV, F3 GeoJSON), y F3 tiene su muestra en /datos/muestras; (3) F1 se regeneró (v2) con los códigos oficiales de estación de F2 para poder integrar las fuentes, conservando sus errores de calidad; (4) la tabla de hechos solo contiene métricas del evento de validación (se retiraron tiempo_espera_estimado_min y monto_descuento_transbordo); (5) la flota se unificó en 450 buses; (6) el diagnóstico BIMM se corrige a fase 2 «Datos accesibles sin BI», porque los datos existen y son accesibles aunque no haya BI.

1. Modelo físico

1.1 Motor y base de datos

Motor: **PostgreSQL 16** (16.x), instalado localmente en el equipo de cada integrante, puerto 5432, codificación UTF8. Base de datos: **movibogota_dw**.

Esquemas: **dw**, con las 6 dimensiones y la tabla de hechos; y **conciliacion**, que guarda una copia cruda de las fuentes (todo como texto) solo para las consultas de la sección 3. No se usa capa de staging: PDI limpia los datos en memoria y escribe directamente en dw. Los scripts están en /dw/ddl (00 crea la base, 01 el esquema, 02 a 08 las tablas y 09 reinicia la carga y crea los miembros «Desconocido»).

1.2 Tablas creadas

Las dimensiones usan llave subrogada SERIAL generada en la carga y conservan la llave de la fuente como atributo único (cod_estacion, cod_ruta, cod_bus). dim_tiempo usa una llave inteligente aaaammdd y dim_hora el minuto del día (0–1.439). Cada dimensión tiene un miembro «Desconocido» con llave −1. Los registros incluyen ese miembro.

Tabla	Tipo	Llave primaria	Llaves foráneas	Registros	Script DDL
dim_tiempo	Dimensión	sk_tiempo	—	731	dw/ddl/02_dim_tiempo.sql
dim_hora	Dimensión	sk_hora	—	1.440	dw/ddl/03_dim_hora.sql
dim_estacion	Dimensión	sk_estacion	—	154	dw/ddl/04_dim_estacion.sql
dim_ruta	Dimensión	sk_ruta	—	8	dw/ddl/05_dim_ruta.sql
dim_bus	Dimensión	sk_bus	—	451	dw/ddl/06_dim_bus.sql
dim_tarifa	Dimensión	sk_tarifa	—	16	dw/ddl/07_dim_tarifa.sql

Tabla	Tipo	Llave primaria	Llaves foráneas	Registros	Script DDL
fact_validaciones	Hechos	id_validacion	sk_tiempo, sk_hora, sk_estacion, sk_ruta, sk_bus, sk_tarifa	54.775	dw/ddl/ 08_fact_validaciones.sql

Granularidad (la misma de la Entrega 1): una validación de pasaje en una estación, ruta, bus y tarifa, en un minuto. Métricas: **cantidad_validaciones** (aditiva, 1 por fila) y **monto_pagado** (aditiva, COP). cod_validacion es una dimensión degenerada. Frente a la Entrega 1, dim_tarjeta se reemplazó por dim_tarifa (F1 no trae identificador de tarjeta) y se agregaron dim_hora y dim_bus.

1.3 Diagrama del modelo físico

El esquema implementado es **en estrella**: la tabla fact_validaciones en el centro y 6 dimensiones desnormalizadas unidas por llaves foráneas. El diagrama se generó desde la base de datos ya creada, leyendo el catálogo information_schema con docs/diagramas/generar_diagrama_fisico.py, y coincide con los scripts DDL. Archivo: /docs/diagramas/modelo_fisico_dw.png.

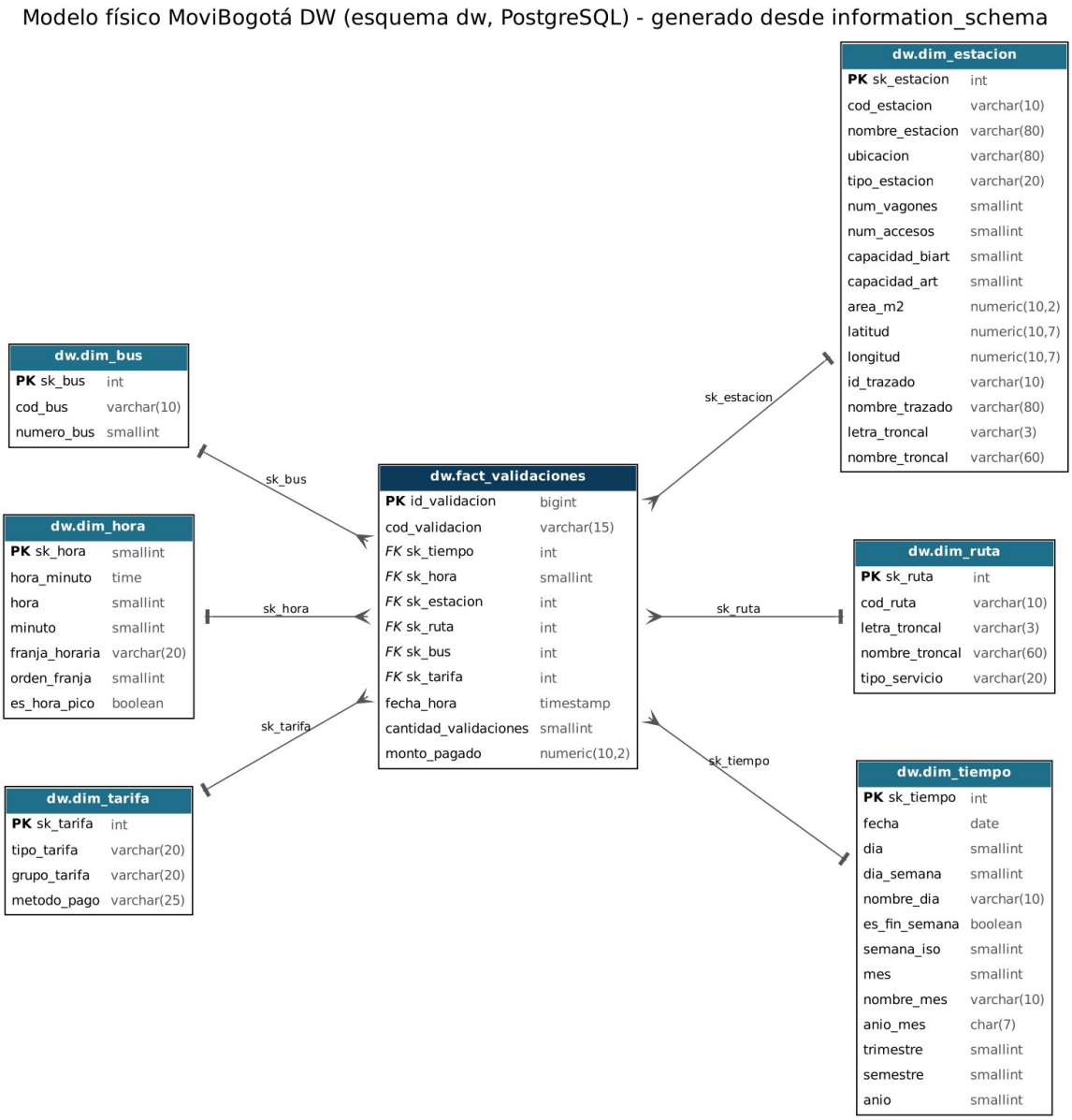


Figura 1. Modelo físico del esquema dw generado desde PostgreSQL.

1.4 Dimensión de tiempo

Rango: del **01/01/2024 al 31/12/2025**, el periodo completo de los hechos. Se pobló con PDI (t_dim_tiempo.ktr: un paso Table input con generate_series entre los parámetros FECHA_INICIO y FECHA_FIN, y un Table output). Registros: **731 días**, sin huecos: la consulta de control da 731 días esperados, 731 cargados y 731 días con validaciones.

Atributos: sk_tiempo, fecha, dia, dia_semana, nombre_dia, es_fin_semana, semana_iso, mes, nombre_mes, anio_mes, trimestre, semestre y anio. Jerarquía: día → mes → trimestre → semestre → año.

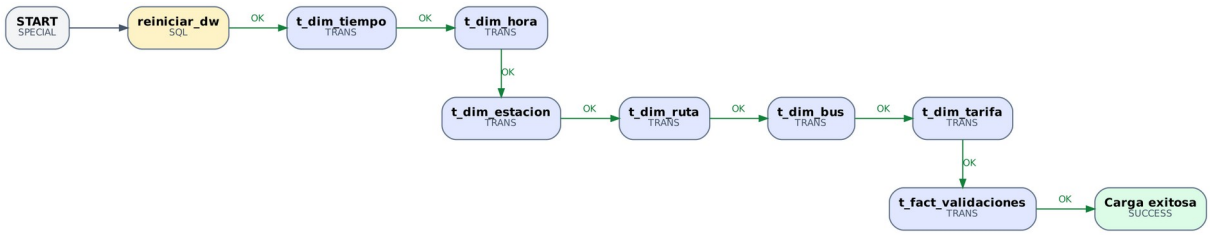
La hora del día se modela en dim_hora (t_dim_hora.ktr, 1.440 minutos), con la jerarquía minuto → hora → franja_horaria (Madrugada, Pico mañana 05:00–08:59, Valle mañana, Mediodía, Pico tarde 16:00–19:59 y Noche) y el indicador es_hora_pico.

2. Procesos ETL

2.1 Trabajo de carga

El trabajo **etl/trabajos/j_carga_dw.kjb** ejecuta en secuencia, y cada paso solo si el anterior terminó bien: START → reiniciar_dw (SQL 09_reiniciar_carga.sql) → t_dim_tiempo → t_dim_hora → t_dim_estacion → t_dim_ruta → t_dim_bus → t_dim_tarifa → **t_fact_validaciones** → Carga exitosa. Las dimensiones se cargan primero y los hechos al final. Las rutas a las fuentes son relativas (${Internal.Entry.Current.Directory}) y la conexión usa variables de kettle.properties, sin contraseñas en el repositorio.

Figura: entradas y saltos de etl/trabajos/j_carga_dw.kjb (diagrama generado a partir del XML del archivo).



2.2 Transformaciones

Leídos = filas del paso de entrada; escritos = filas del Table output; rechazos = filas enviadas a /etl/rechazos. Valores de las métricas de pasos de la ejecución completa.

Archivo .ktr	Fuente(s) y formato	Destino	Pasos principales	Leídos	Escritos	Rechazos
t_dim_tiempo.ktr	Generada (calendario)	dim_tiempo	Table input genera_calendario → Table output	731	731	0
t_dim_hora.ktr	Generada (minutos del día)	dim_hora	Table input genera_minutos_dia → Table output	1.440	1.440	0
t_dim_estacion.ktr	F2 (CSV) + F3 (GeoJSON)	dim_estacion	CSV file input → Filter rows + JavaScript regla_* → Stream lookup busca_trazado_F3 (← JSON input) → Table output	153 + 22	153	0
t_dim_ruta.ktr	F1 (CSV) + F3 (GeoJSON)	dim_ruta	CSV file input → Unique rows (HashSet) → JavaScript → Stream lookup busca_troncal_F3 (← JSON input) → Table output	55.000 + 22	7	0
t_dim_bus.ktr	F1 (CSV)	dim_bus	CSV file input → Filter rows bus_informado → JavaScript → Unique rows (HashSet) → Table output	55.000	450	0

Archivo .ktr	Fuente(s) y formato	Destino	Pasos principales	Leídos	Escritos	Rechazos
t_dim_tarifa.ktr	F1 (CSV)	dim_tarifa	CSV file input → Unique rows (HashSet) → JavaScript → Table output	55.000	15	0
t_fact_validaciones.ktr	F1 (CSV)	fact_validaciones	CSV file input → Unique rows (rechazo) → Filter rows + JavaScript (fechas, estación, bus) → Filter rows monto → 4 Database lookup → Table output	55.000	54.775	225

2.3 Reglas de limpieza aplicadas

Las tres primeras filas corresponden a los problemas de la tabla 2.2 de la Entrega 1. En la Entrega 1 los problemas de id_estacion e id_bus se atribuyeron a F2 y F3, pero ambos campos están en F1. Las demás filas son problemas detectados al perfilar las fuentes. Registros afectados = filas que pasan por el paso de la regla según las métricas de pasos.

Problema (Entrega 1)	Fuente y campo	Regla aplicada	Paso de PDI (nombre)	Registros afectados
Formatos de fecha heterogéneos (12 h AM/PM)	F1.fecha_hora	Convertir dd/mm/aaaa hh:mm AM|PM a fecha y hora de 24 h	regla_normaliza_fecha_ampm (JavaScript)	5.499
Formatos de fecha heterogéneos (epoch Unix)	F1.fecha_hora	Convertir segundos epoch (UTC) a la hora local de Bogotá (UTC−5)	regla_normaliza_fecha_epoch (JavaScript)	2.656
Espacios y minúsculas en el código de estación	F1.id_estacion	Quitar espacios y pasar a mayúsculas (« est-07103 » → «EST-07103»)	regla_normaliza_codigo_estacion (JavaScript)	2.230
Bus vacío por falla de transmisión	F1.id_bus	Asignar SIN-BUS (llave −1 en dim_bus); no crea miembro en dim_bus	regla_bus_vacio_a_desconocido (JavaScript)	1.107
Validaciones duplicadas	F1.id_validacion	Conservar la primera aparición y enviar las repetidas a rechazos_duplicados.csv	regla_elimina_duplicados (Unique rows by HashSet)	180
Montos en cero o negativos	F1.monto_pagado	Descartar monto ≤ 0 a rechazos_monto_invalido.csv	regla_valida_monto_positivo (Filter rows)	45
Estación no catalogada	F1.id_estacion	Sin coincidencia en dim_estacion: cargar con la llave −1 (Desconocida)	regla_estacion_desconocida (Filter rows + Database lookup)	38
Código sin cero a la izquierda	F2.num_est	Completar a 5 dígitos (9005 → 09005)	regla_codigo_5_digitos (JavaScript)	4
Guion largo, «AV.» y espacios sobrantes	F2.nom_est	Normalizar a «-», «Av.» y un solo espacio	regla_normaliza_nombre_estacion (JavaScript)	17
Área 0 (dato no medido)	F2.area_est	Reemplazar 0 por nulo	regla_area_cero_a_nulo (JavaScript)	17

Problema (Entrega 1)	Fuente y campo	Regla aplicada	Paso de PDI (nombre)	Registros afectados
Nombres sin tildes y mayúsculas irregulares	F3.nom_traz, F3.nom_tronc	Corregir Americas → Américas, Extension → Extensión, De → de	regla_estandariza_nombres_troncal (JavaScript)	7 (de 22 procesados)

2.4 Evidencia de ejecución

Resultado de la carga completa sobre una base vacía (09_reiniciar_carga.sql y luego las 7 cargas): filas por tabla del DW, consultadas en el esquema dw después de la ejecución. De las 55.000 filas de F1 se escriben 54.775 en fact_validaciones; las 225 restantes quedan en /etl/rechazos.

Figura: filas cargadas por tabla en movibogota_dw.

Tabla	Filas	Explicación
dw.dim_tiempo	731	730 días 2024–2025 + miembro −1
dw.dim_hora	1.440	1.440 minutos del día
dw.dim_estacion	154	153 estaciones de F2 + miembro −1
dw.dim_ruta	8	7 rutas + miembro −1
dw.dim_bus	451	450 buses + miembro −1
dw.dim_tarifa	16	tarifas × método de pago + miembro −1
dw.fact_validaciones	54.775	55.000 leídas − 180 duplicadas − 45 montos ≤ 0

3. Conciliación

3.1 Conteo por fuente

Consultas: /dw/conciliacion/02_conteos_por_fuente.sql (Anexo A). En todas las fuentes leídos = cargados + descartados.

Fuente	Leídos	Cargados	Descartados	Motivo de los descartes
F1	55.000	54.775	225	180 validaciones duplicadas (mismo id_validacion) y 45 con monto ≤ 0. Además, 38 filas se cargaron con estación −1 y 1.106 con bus −1: se conservan, no son descartes.
F2	153	153	0	Sin descartes: las 153 estaciones están en dim_estacion (154 filas con el miembro −1).
F3	22	17	5	Los trazados TZ004, TZ006, TZ017, TZ020 y TZ022A no tienen estaciones en F2, así que no aparecen en dim_estacion. Las 13 letras de troncal de F3 sí se usan para nombrar las 7 rutas de dim_ruta.

3.2 Totales de una métrica

Consulta: /dw/conciliacion/03_totales_metrica.sql (Anexo A).

Métrica	Total en la fuente	Total en el DW	Diferencia y explicación
monto_pagado (COP)	$159.508.900	$159.054.300	$454.600 = $524.950 de las 180 validaciones duplicadas − $70.350 de los 45 montos ≤ 0 descartados (los negativos restaban en la fuente).
cantidad_validaciones	55.000	54.775	225 = 180 duplicados + 45 montos ≤ 0.

4. Tablero

4.1 Indicadores

Campos calculados de Looker Studio sobre la vista dw.v_tablero_validaciones (fact_validaciones unida a sus 6 dimensiones). Detalle en /dashboard/README.md. Valores sin filtros (periodo 2024–2025).

Indicador	Fórmula (DAX o SQL)	Qué mide	Referencia	Pregunta
Validaciones del último año	Validaciones = SUM(cantidad_validaciones) Variación: scorecard con periodo 2025 comparado con el año anterior	Demanda de pasajeros	Periodo anterior: 28.291 en 2025 frente a 26.484 en 2024 (+6,8 %)	P0, P1
Recaudo (COP) del último año	Recaudo COP = SUM(monto_pagado); scorecard 2025 comparado con el año anterior	Ingreso por pasajes	Periodo anterior: $85.264.100 en 2025 frente a $73.790.200 en 2024 (+15,5 %)	P0
% de validaciones en hora pico	SUM(CASE WHEN es_hora_pico THEN cantidad_validaciones ELSE 0 END) / SUM(cantidad_validaciones)	Concentración de la demanda en 05:00–08:59 y 16:00–19:59	Meta ≤ 55 %: resultado 56,6 % (sobre la meta)	P0, P2
Validaciones por bus en servicio	SUM(cantidad_validaciones) / COUNT_DISTINCT(cod_bus)	Carga de demanda que atiende cada bus asignado a una ruta	Promedio de la red: 121,7 (B12 = 163,6; L10 = 93,2)	P2

4.2 Respuesta a las preguntas de negocio

Por la retroalimentación de la Entrega 1, las preguntas de la tabla 1.3 se ajustaron a los datos disponibles, manteniendo la decisión de negocio (redistribuir flota y frecuencias). P0 «¿Qué troncales, estaciones y franjas horarias concentran la mayor demanda de validaciones y recaudo, por mes, en 2024–2025?»; P1 «¿Cuál es el volumen diario de validaciones por tipo de tarifa y estación?»; P2 «¿Qué rutas tienen más demanda en hora pico frente a valle y cuántas validaciones atiende cada bus asignado?». P3 (pronóstico de demanda por estación y franja) es de tipo BA y se desarrolla en la Entrega 3.

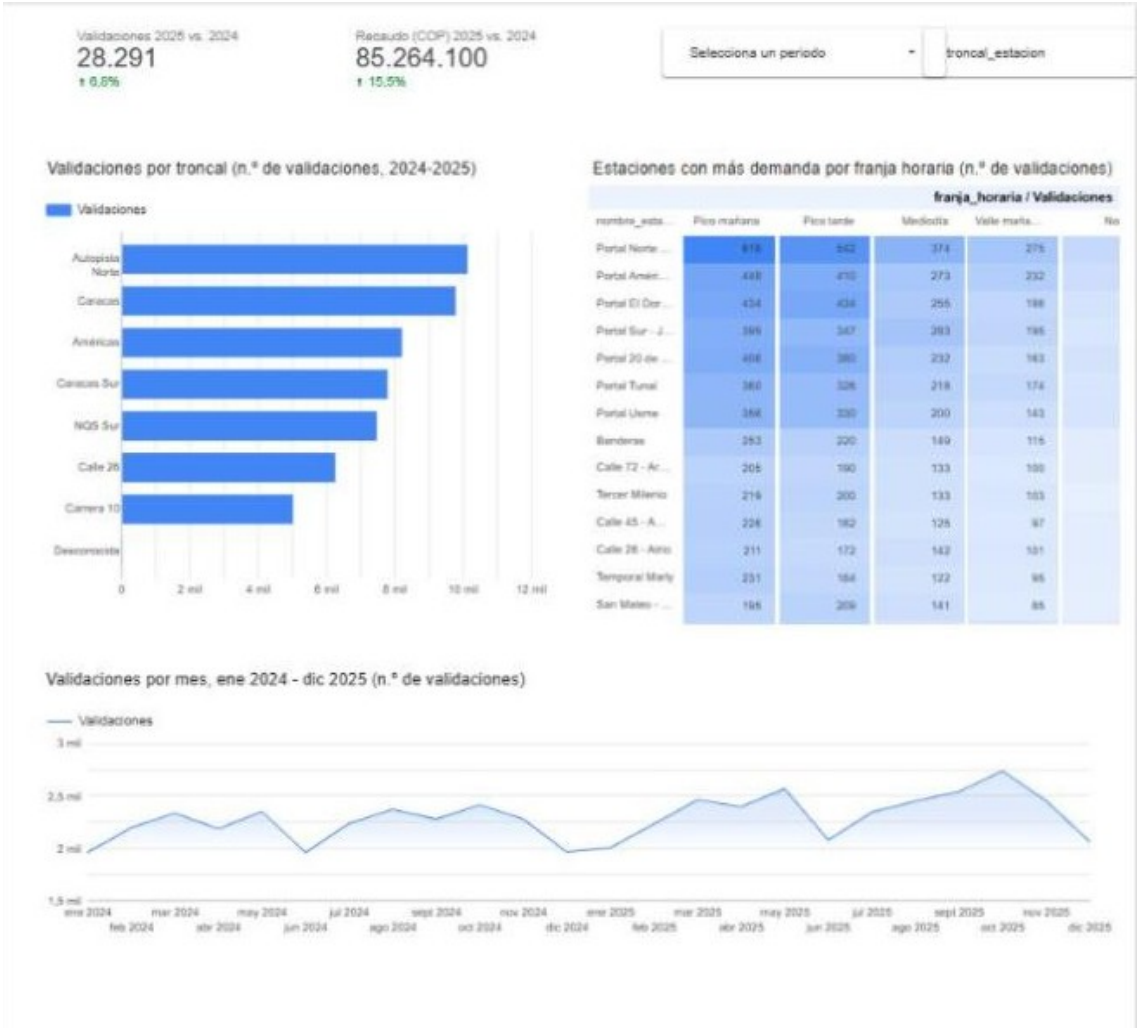
Pregunta	Página y visual	Tipo de gráfico y por qué	Respuesta encontrada (con cifra)
P0	Página 1 «Demanda por estación y franja»: barras por troncal, matriz Top 10 estaciones × franja y línea mensual	Barras horizontales ordenadas para comparar troncales; matriz con escala de color para cruzar estación y franja; línea para la tendencia mensual	Autopista Norte lidera con 10.141 validaciones (18,5 %) y $29.442.500, seguida de Caracas (9.791; 17,9 %). Portal Norte - Unicervantes es la estación con más demanda (2.058), con su máximo en el pico de la mañana (618). Los picos suman el 56,6 % (mañana 29,4 %, tarde 27,2 %). Mes más alto: octubre de 2025 (2.733); más bajo: junio de 2024 (1.956).

Pregunta	Página y visual	Tipo de gráfico y por qué	Respuesta encontrada (con cifra)
P1	Página 2 «Volumen diario por tarifa y estación»: barras de promedio diario por tarifa y matriz estación × tarifa	Barras ordenadas para comparar 5 tarifas; matriz para el detalle por estación; serie mensual para comparar tarifa plena y subsidiada	La red registra 74,9 validaciones por día: General 45,7 (61,0 %), Sisbén 11,1 (14,9 %), Adulto Mayor 10,5 (14,1 %), Estudiantil 3,8 (5,1 %) y Discapacidad 3,7 (4,9 %). El 39,0 % usa tarifa subsidiada. Portal Norte registra 2,8 validaciones por día, 1.263 de ellas con tarifa General.
P2	Página 3 «Rutas y flota: pico vs. valle»: barras agrupadas pico/valle por ruta y barras de validaciones por bus con línea del promedio	Barras agrupadas para comparar dos medidas por ruta; barras ordenadas con línea de referencia para ver qué ruta supera el promedio; columnas por hora para el perfil diario	B12 (Autopista Norte) tiene la mayor demanda: 5.730 validaciones en pico y 4.416 en valle, y atiende 163,6 validaciones por bus, 34 % sobre el promedio de la red (121,7). K16 (98,0) y L10 (93,2) están por debajo, así que conviene trasladar buses de K16 y L10 a B12 en las franjas pico. La hora de mayor demanda en días hábiles es 06:00 (4.740).

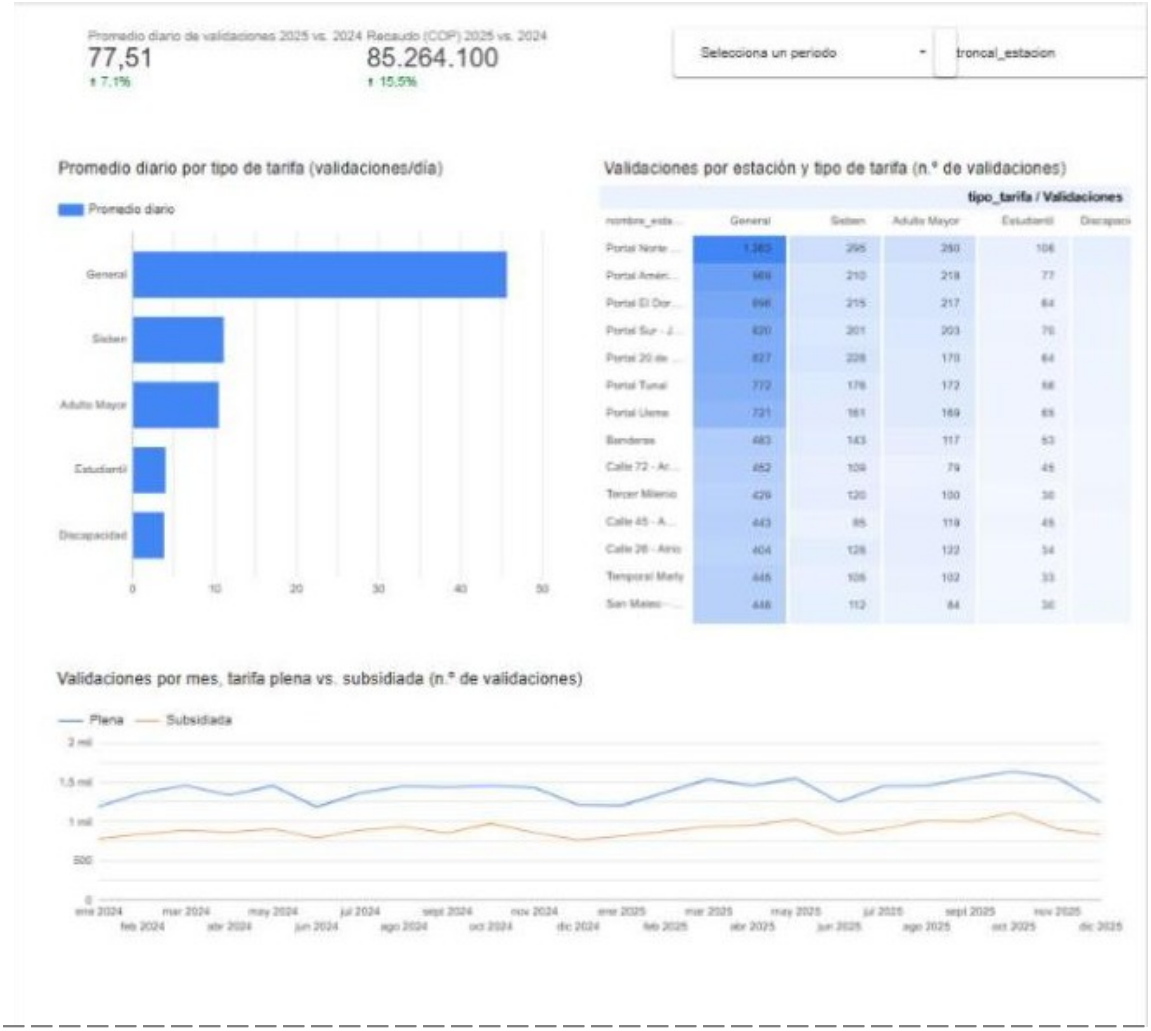
4.3 Capturas del tablero

Capturas de las tres páginas del tablero en Looker Studio (https://datastudio.google.com/reporting/ff362d1b-b283-4d16-969c-23989bdc0614), conectado con el conector PostgreSQL a una copia del DW movibogota_dw (vista dw.v_tablero_validaciones, usuario de solo lectura). Las mismas imágenes van en /dashboard.

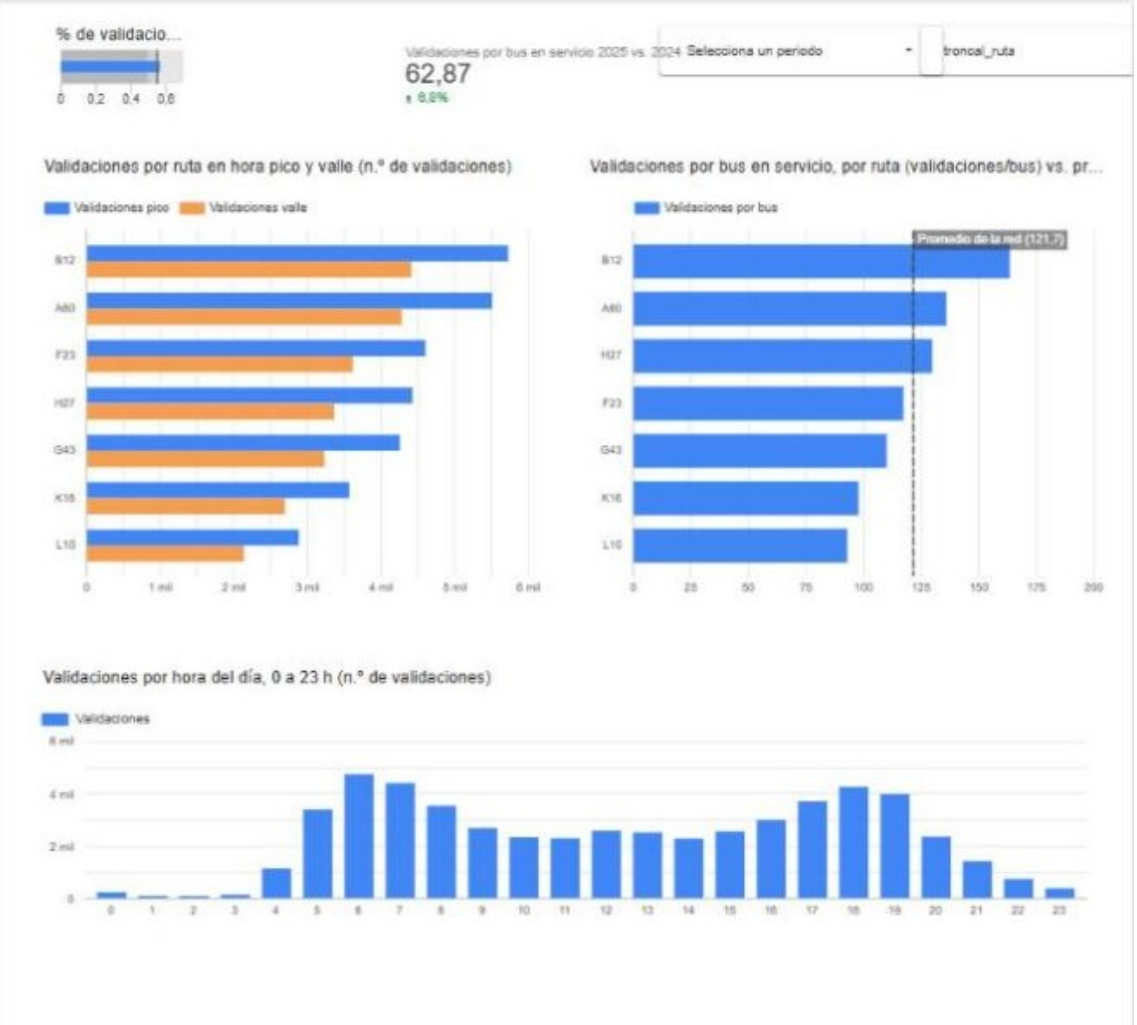
Página 1 (P0) · Demanda por estación y franja



Página 2 (P1) · Volumen diario por tarifa y estación


Promedio diario de validaciones 2025 vs. 2024
77,51
↑ 7,1%
Recaudo (COP) 2025 vs. 2024
85.264.100
↑ 15,5%
Selecciona un periodo
troncal_estacion
Promedio diario por tipo de tarifa (validaciones/día)
Promedio diario
General
Sisben
Adulto Mayor
Estudianti
Discapacidad
0
10
20
30
40
50
Validaciones por estación y tipo de tarifa (n.° de validaciones)
tipo_tarifa / Validaciones
nombre_esta...
General
Sisben
Adulto Mayor
Estudianti
Discapaci
Portal Norte ...
1.362
295
260
106
Portal Améri...
669
210
218
77
Portal El Dor...
696
215
217
64
Portal Sur - J...
620
201
203
70
Portal 20 de ...
627
228
170
64
Portal Tunal
772
176
172
68
Portal Usme
721
161
169
63
Banderas
483
143
117
53
Calle 72 - Ar...
462
109
79
45
Tercer Milenio
429
120
100
30
Calle 45 - A...
443
85
119
45
Calle 26 - Aero
404
128
122
54
Temporal Marly
446
105
102
33
San Mateo - ...
448
112
84
30
Validaciones por mes, tarifa plena vs. subsidiada (n.° de validaciones)
Plena
Subsidiada
2 mil
1,5 mil
1 mil
500
0
ene 2024
feb 2024
mar 2024
abr 2024
may 2024
jun 2024
jul 2024
ago 2024
sept 2024
oct 2024
nov 2024
dic 2024
ene 2025
feb 2025
mar 2025
abr 2025
may 2025
jun 2025
jul 2025
ago 2025
sept 2025
oct 2025
nov 2025
dic 2025

Página 3 (P2) · Rutas y flota: pico vs. valle



4.4 Filtros

Controles disponibles (por defecto filtran todos los visuales de su página):

- **Periodo** (control de intervalo de fechas sobre dim_tiempo.fecha): páginas 1, 2 y 3.
- **Troncal de la estación** (lista desplegable, dim_estacion.nombre_troncal): páginas 1 y 2.
- **Troncal de la ruta** (lista desplegable, dim_ruta.nombre_troncal): página 3.

Anexos

Anexo A. Consultas de conciliación

Consultas de /dw/conciliacion. 01_cargar_fuentes_crudas.sql copia las fuentes sin limpiar al esquema conciliacion; 02 y 03 producen las tablas 3.1 y 3.2; 04_cifras_tablero.sql verifica las cifras del tablero.

01_cargar_fuentes_crudas.sql

```
-- ======================================================================
-- Conciliación · 01 · Copia cruda de las fuentes (solo para contar y sumar)
-- Ejecutar con psql DESDE LA RAÍZ DEL REPOSITORIO:
--   psql -h localhost -U postgres -d movibogota_dw -f dw/conciliacion/01_cargar_fuentes_crudas.sql
```

```
-- Las fuentes se copian tal cual (todo como texto, sin limpiar) en el esquema conciliacion.
-- ========================================================================
DROP SCHEMA IF EXISTS conciliacion CASCADE;
CREATE SCHEMA conciliacion;

CREATE TABLE conciliacion.f1_validaciones (
    id_validacion TEXT, fecha_hora TEXT, id_estacion TEXT, id_ruta TEXT,
    id_bus TEXT, tipo_tarifa TEXT, monto_pagado TEXT, metodo_pago TEXT);
\copy conciliacion.f1_validaciones FROM 'datos/raw/Dataset_registros.csv' WITH (FORMAT csv, HEADER
true, ENCODING 'UTF8')

CREATE TABLE conciliacion.f2_estaciones (
    x TEXT, y TEXT, observ TEXT, longitud TEXT, latitud TEXT, objectid TEXT, num_est TEXT, nom_est
TEXT,
    id_trazado TEXT, long_est TEXT, ancho_est TEXT, area_est TEXT, esta_oper TEXT, eta_oper TEXT,
    tipo_esta TEXT, ub_est TEXT, num_acc TEXT, acc_esp TEXT, acc_depr TEXT, acc_puent TEXT, cod_nodo
TEXT,
    cap_biart TEXT, num_vag TEXT, cap_art TEXT, globalid TEXT, created_user TEXT, created_date TEXT,
    last_edited_user TEXT, last_edited_date TEXT);
\copy conciliacion.f2_estaciones FROM 'datos/raw/Estaciones_Troncales_de_TRANSMILENIO.csv' WITH (FORMAT
csv, HEADER true, ENCODING 'UTF8')

-- F3 (GeoJSON): una línea por objeto Feature; se lee línea a línea y se interpreta como JSON
CREATE TABLE conciliacion.f3_lineas (linea TEXT);
\copy conciliacion.f3_lineas FROM 'datos/raw/Trazados_Troncales_de_TRANSMILENIO.geojson' WITH (FORMAT
text, DELIMITER E'\x01', ENCODING 'UTF8')
CREATE TABLE conciliacion.f3_trazados AS
SELECT (regexp_replace(linea, ',\s*$', ''))::jsonb -> 'properties' ->> 'id_trazado' AS id_trazado,
       (regexp_replace(linea, ',\s*$', ''))::jsonb -> 'properties' ->> 'nom_tronc'  AS nom_tronc
FROM conciliacion.f3_lineas
WHERE linea LIKE '{"type":"Feature"%';
```

02_conteos_por_fuente.sql

```
-- ========================================================================
-- Conciliación · 02 · Leídos = cargados + descartados, por fuente (tabla 3.1)
-- Requiere haber ejecutado 01_cargar_fuentes_crudas.sql
-- ========================================================================

-- F1 · validaciones -> dw.fact_validaciones
WITH fuente AS (
    SELECT count(*)                       AS leidos,
           count(*) - count(DISTINCT id_validacion) AS duplicados
    FROM conciliacion.f1_validaciones),
unicas AS (   -- primera aparición de cada id, como en el paso regla_elimina_duplicados
    SELECT DISTINCT ON (id_validacion) * FROM conciliacion.f1_validaciones),
monto AS (
    SELECT count(*) AS monto_no_valido FROM unicas WHERE monto_pagado::numeric <= 0),
dw AS (
    SELECT count(*) AS cargados FROM dw.fact_validaciones)
SELECT 'F1' AS fuente, f.leidos, d.cargados, f.duplicados + m.monto_no_valido AS descartados,
       f.duplicados, m.monto_no_valido,
       f.leidos = d.cargados + f.duplicados + m.monto_no_valido AS cuadra
FROM fuente f, monto m, dw d;

-- F2 · estaciones -> dw.dim_estacion (sin el miembro -1)
SELECT 'F2' AS fuente,
       (SELECT count(*) FROM conciliacion.f2_estaciones)               AS leidos,
       (SELECT count(*) FROM dw.dim_estacion WHERE sk_estacion > 0)    AS cargados,
       (SELECT count(*) FROM conciliacion.f2_estaciones)
     - (SELECT count(*) FROM dw.dim_estacion WHERE sk_estacion > 0)    AS descartados;

-- F3 · trazados -> atributos de trazado/troncal en dw.dim_estacion y dw.dim_ruta
SELECT 'F3' AS fuente,
       (SELECT count(*) FROM conciliacion.f3_trazados)                              AS leidos,
       (SELECT count(*) FROM conciliacion.f3_trazados t
         WHERE EXISTS (SELECT 1 FROM dw.dim_estacion e WHERE e.id_trazado = t.id_trazado)) AS cargados,
       (SELECT count(*) FROM conciliacion.f3_trazados t
         WHERE NOT EXISTS (SELECT 1 FROM dw.dim_estacion e WHERE e.id_trazado = t.id_trazado)) AS
descartados,
       (SELECT string_agg(id_trazado, ', ' ORDER BY id_trazado) FROM conciliacion.f3_trazados t
         WHERE NOT EXISTS (SELECT 1 FROM dw.dim_estacion e WHERE e.id_trazado = t.id_trazado)) AS
trazados_sin_estacion;

-- Detalle: registros cargados con llave «Desconocido» (-1)
```

```
SELECT 'sk_estacion = -1' AS caso, count(*) FROM dw.fact_validaciones WHERE sk_estacion = -1
UNION ALL SELECT 'sk_bus = -1', count(*) FROM dw.fact_validaciones WHERE sk_bus = -1
UNION ALL SELECT 'sk_ruta = -1', count(*) FROM dw.fact_validaciones WHERE sk_ruta = -1
UNION ALL SELECT 'sk_tarifa = -1', count(*) FROM dw.fact_validaciones WHERE sk_tarifa = -1;

-- Conteo de filas de cada tabla del DW (tabla 1.2)
SELECT 'dim_tiempo' AS tabla, count(*) FROM dw.dim_tiempo
UNION ALL SELECT 'dim_hora', count(*) FROM dw.dim_hora
UNION ALL SELECT 'dim_estacion', count(*) FROM dw.dim_estacion
UNION ALL SELECT 'dim_ruta', count(*) FROM dw.dim_ruta
UNION ALL SELECT 'dim_bus', count(*) FROM dw.dim_bus
UNION ALL SELECT 'dim_tarifa', count(*) FROM dw.dim_tarifa
UNION ALL SELECT 'fact_validaciones', count(*) FROM dw.fact_validaciones;
```

03_totales_metrica.sql

```
-- ======================================================================
-- Conciliación · 03 · Total de métricas fuente vs. DW (tabla 3.2)
-- ======================================================================
WITH f AS (SELECT * FROM conciliacion.f1_validaciones),
rn AS (SELECT *, row_number() OVER (PARTITION BY id_validacion ORDER BY ctid) AS n
           FROM conciliacion.f1_validaciones)   -- los duplicados son copias exactas
SELECT 'monto_pagado (COP)'                                                 AS metrica,
       (SELECT sum(monto_pagado::numeric) FROM f)                           AS total_fuente,
       (SELECT sum(monto_pagado) FROM dw.fact_validaciones)                 AS total_dw,
       (SELECT sum(monto_pagado::numeric) FROM f)
     - (SELECT sum(monto_pagado) FROM dw.fact_validaciones)                 AS diferencia,
       (SELECT sum(monto_pagado::numeric) FROM rn WHERE n > 1)              AS monto_duplicados,
       (SELECT sum(monto_pagado::numeric) FROM rn WHERE n = 1 AND monto_pagado::numeric <= 0) AS
monto_no_valido
UNION ALL
SELECT 'cantidad_validaciones',
       (SELECT count(*) FROM f),
       (SELECT sum(cantidad_validaciones) FROM dw.fact_validaciones),
       (SELECT count(*) FROM f) - (SELECT sum(cantidad_validaciones) FROM dw.fact_validaciones),
       (SELECT count(*) FROM rn WHERE n > 1),
       (SELECT count(*) FROM rn WHERE n = 1 AND monto_pagado::numeric <= 0);

-- Control por año: el total debe coincidir también por periodo
SELECT t.anio, count(*) AS validaciones, sum(f.monto_pagado) AS recaudo
FROM dw.fact_validaciones f JOIN dw.dim_tiempo t USING (sk_tiempo)
GROUP BY t.anio ORDER BY t.anio;

-- Dimensión de tiempo sin huecos: días esperados vs. días cargados entre la primera y la última
validación
SELECT min(fecha) AS desde, max(fecha) AS hasta,
       (max(fecha) - min(fecha) + 1) AS dias_esperados, count(*) AS dias_en_dim_tiempo,
       (SELECT count(DISTINCT sk_tiempo) FROM dw.fact_validaciones) AS dias_con_validaciones
FROM dw.dim_tiempo;
```

Anexo B. Capturas adicionales

Figura B.1: pasos y saltos de etl/transformaciones/t_dim_estacion.ktr (generado a partir del XML del archivo).

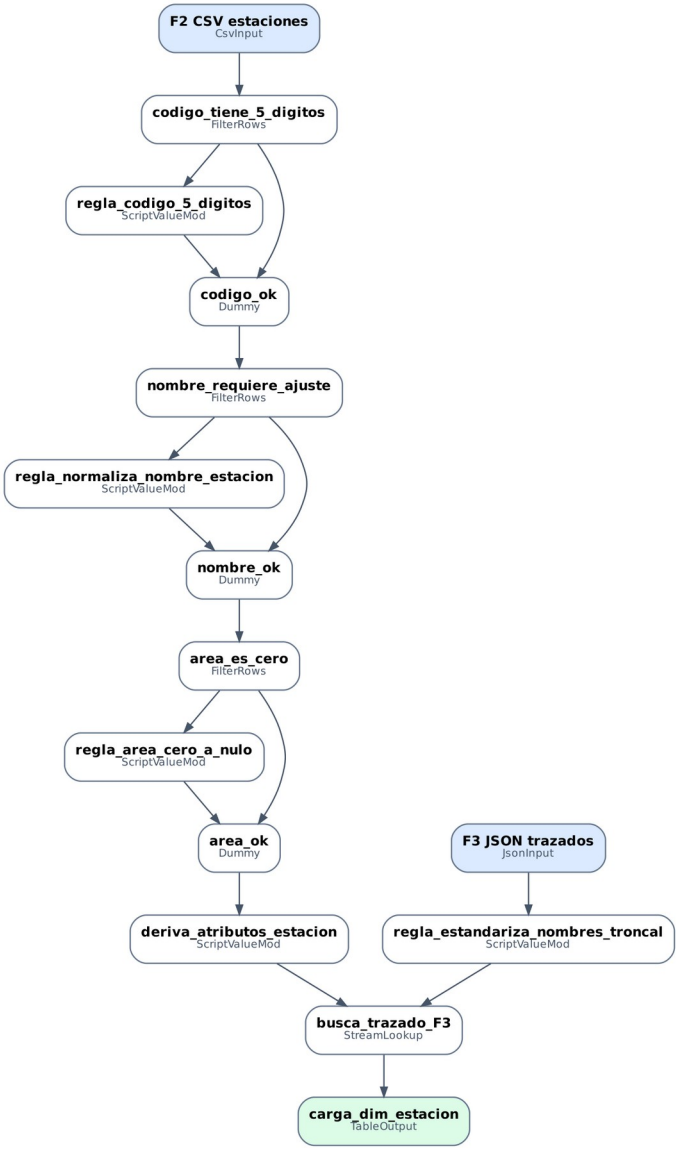
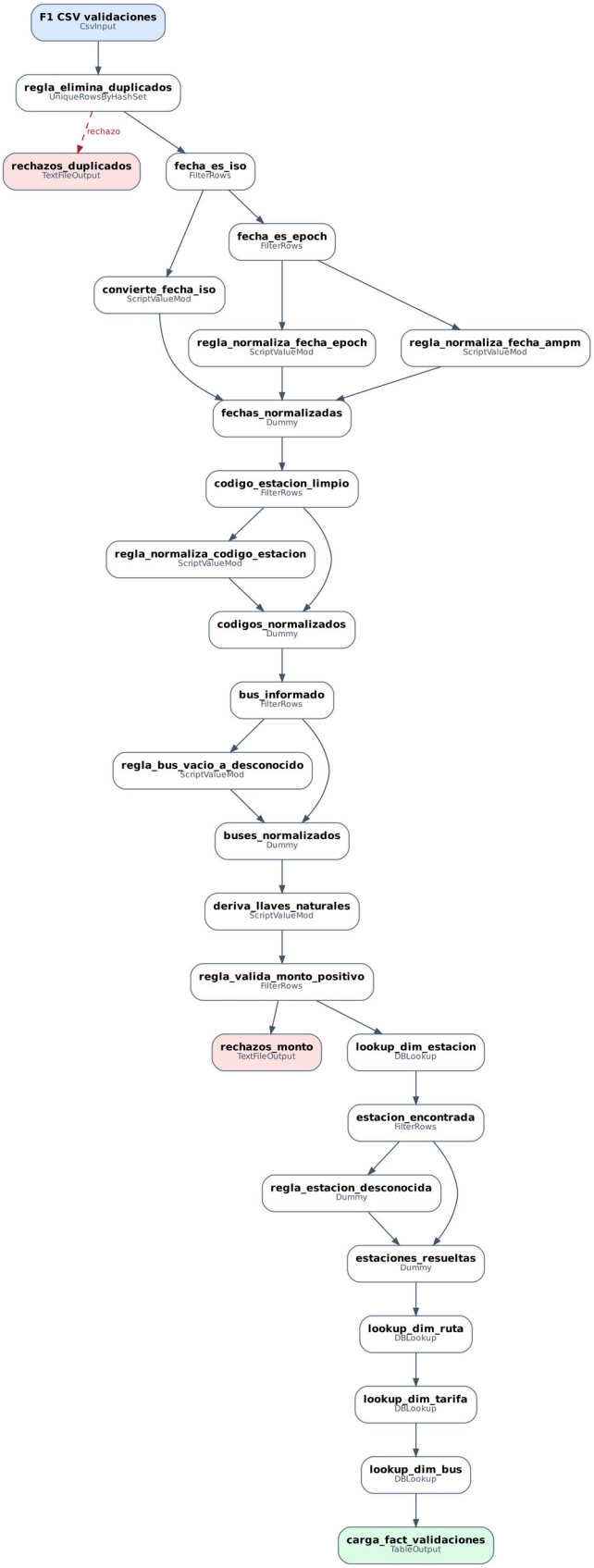


Figura B.2: pasos y saltos de etl/transformaciones/t_fact_validaciones.ktr; la línea punteada roja es el flujo de rechazo de

duplicados.


F1 CSV validaciones
CsvInput
regla_elimina_duplicados
UniqueRowsByHashSet
rechazo
rechazos_duplicados
TextFileOutput
fecha_es_iso
FilterRows
fecha_es_epoch
FilterRows
convierte_fecha_iso
ScriptValueMod
regla_normaliza_fecha_epoch
ScriptValueMod
regla_normaliza_fecha_ampm
ScriptValueMod
fechas_normalizadas
Dummy
codigo_estacion_limpio
FilterRows
regla_normaliza_codigo_estacion
ScriptValueMod
codigos_normalizados
Dummy
bus_informado
FilterRows
regla_bus_vacio_a_desconocido
ScriptValueMod
buses_normalizados
Dummy
deriva_llaves_naturales
ScriptValueMod
regla_valida_monto_positivo
FilterRows
rechazos_monto
TextFileOutput
lookup_dim_estacion
DBLookup
estacion_encontrada
FilterRows
regla_estacion_desconocida
Dummy
estaciones_resueltas
Dummy
lookup_dim_ruta
DBLookup
lookup_dim_tarifa
DBLookup
lookup_dim_bus
DBLookup
carga_fact_validaciones
TableOutput